capstone

November 18, 2024

[40]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from prophet import Prophet
from prophet.plot import plot_plotly, plot_components_plotly
from prophet.diagnostics import cross_validation
from prophet.plot import plot_cross_validation_metric
from prophet.diagnostics import performance_metrics

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import RandomizedSearchCV,train_test_split
from sklearn.metrics import␣
↪classification_report,confusion_matrix,ConfusionMatrixDisplay,roc_curve,precision_score,rec
from sklearn.metrics import mean_squared_error as mse
from xgboost import XGBRegressor
import xgboost as xgb
from xgboost import plot_importance
from tabulate import tabulate

import warnings
warnings.simplefilter('ignore')
```

[41]:
```
df=pd.read_csv('/content/price_optimsation_dataset.csv')
```

[42]:
```
df.shape
```

[42]: (676, 30)

0.1 676 rows and 30 columns

[43]:
```
df.isna().sum().sum()
```

[43]: 0

0.2 No missing values

[44]:
```
df[df.duplicated()].shape
```

[44]: (0, 30)

0.3 No duplicated values

[45]:
```
pd.set_option('display.max_columns', None)
df.head()
```

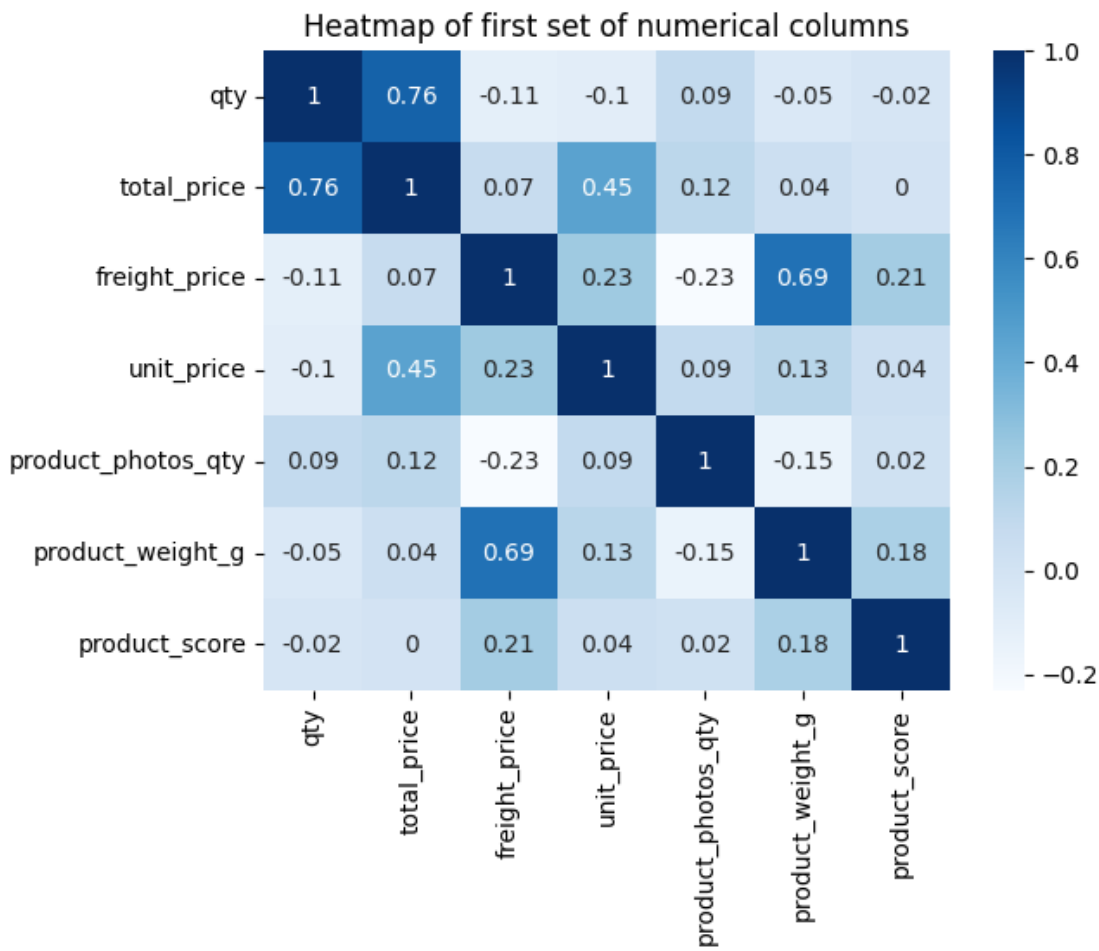
[45]:
```
  product_id product_category_name  month_year  qty  total_price  \
0       bed1        bed_bath_table  01-05-2017    1        45.95
1       bed1        bed_bath_table  01-06-2017    3       137.85
2       bed1        bed_bath_table  01-07-2017    6       275.70
3       bed1        bed_bath_table  01-08-2017    4       183.80
4       bed1        bed_bath_table  01-09-2017    2        91.90

   freight_price  unit_price  product_name_lenght  product_description_lenght  \
0      15.100000       45.95                   39                         161
1      12.933333       45.95                   39                         161
2      14.840000       45.95                   39                         161
3      14.287500       45.95                   39                         161
4      15.100000       45.95                   39                         161

   product_photos_qty  product_weight_g  product_score  customers  weekday  \
0                   2               350            4.0         57       23
1                   2               350            4.0         61       22
2                   2               350            4.0        123       21
3                   2               350            4.0         90       23
4                   2               350            4.0         54       21

   weekend  holiday  month  year          s  volume  comp_1  ps1        fp1  \
0        8        1      5  2017  10.267394    3800    89.9  3.9  15.011897
1        8        1      6  2017   6.503115    3800    89.9  3.9  14.769216
2       10        1      7  2017  12.071651    3800    89.9  3.9  13.993833
3        8        1      8  2017   9.293873    3800    89.9  3.9  14.656757
4        9        1      9  2017   5.555556    3800    89.9  3.9  18.776522

       comp_2  ps2        fp2  comp_3  ps3        fp3  lag_price
0  215.000000  4.4   8.760000   45.95  4.0  15.100000      45.90
1  209.000000  4.4  21.322000   45.95  4.0  12.933333      45.95
2  205.000000  4.4  22.195932   45.95  4.0  14.840000      45.95
3  199.509804  4.4  19.412885   45.95  4.0  14.287500      45.95
4  163.398710  4.4  24.324687   45.95  4.0  15.100000      45.95
```

[46]:
```
set1=['qty','total_price', 'freight_price', 'unit_price', 'product_photos_qty',␣
↪'product_weight_g',
       'product_score']

set2=['product_score', 'customers', 'weekday', 'weekend', 'holiday', 'month',
       'year', 's', 'volume']

set3=['comp_1', 'ps1', 'fp1', 'lag_price', 'total_price', 'qty']

set4=['comp_2', 'ps2', 'fp2', 'lag_price', 'total_price', 'qty']

set5=['comp_3', 'ps3', 'fp3', 'lag_price', 'total_price', 'qty']
```

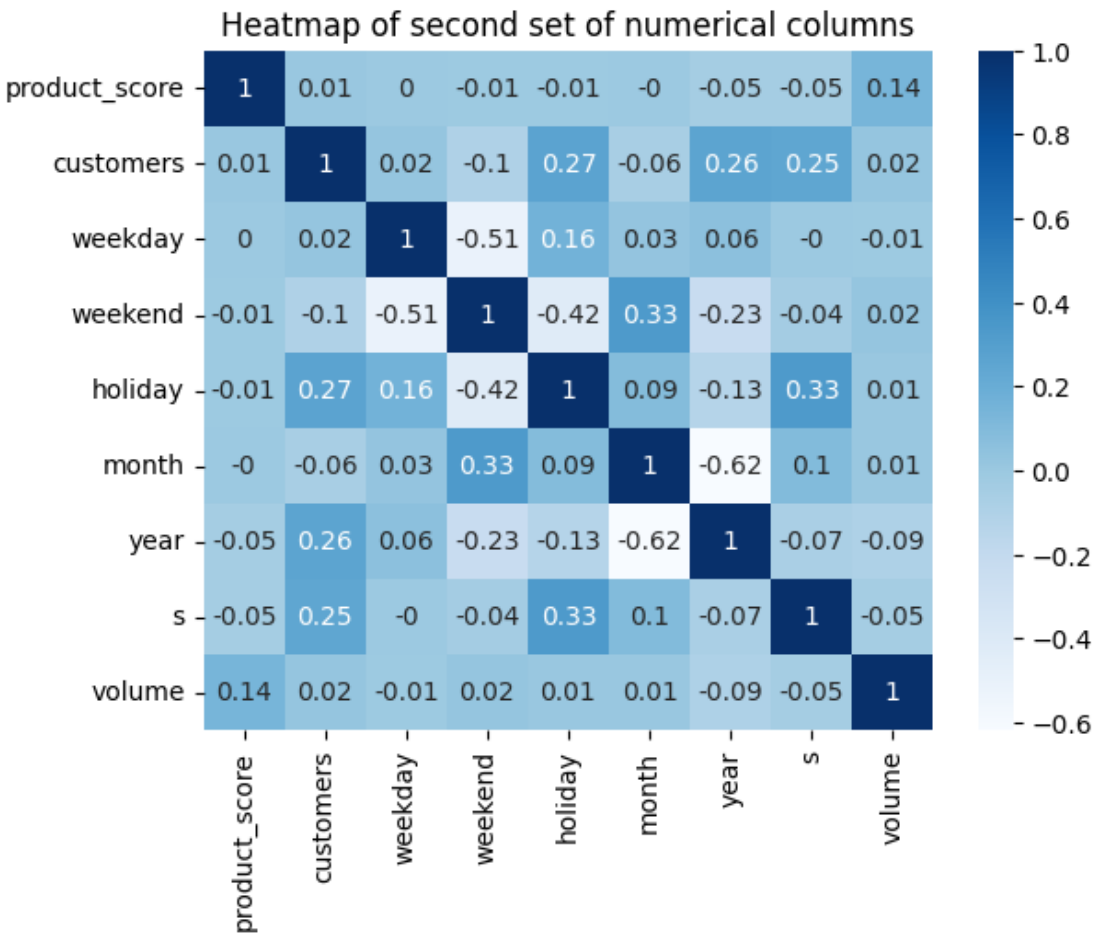
0.4 Outlier clipping

[47]:
```
### Used 2.7 std around the  mean as cutoff for clipping numerical columns

def outlier_clipping(df,set):
    for col in set:
        m,s=np.mean(df[col]),np.std(df[col],ddof=1)
        max_,min_=m+(2.7)*s,m-(2.7)*s
        df[col]=np.clip(df[col],min_,max_)

for set in [set1, set2, set3, set4, set5]:
    outlier_clipping(df,set)
```

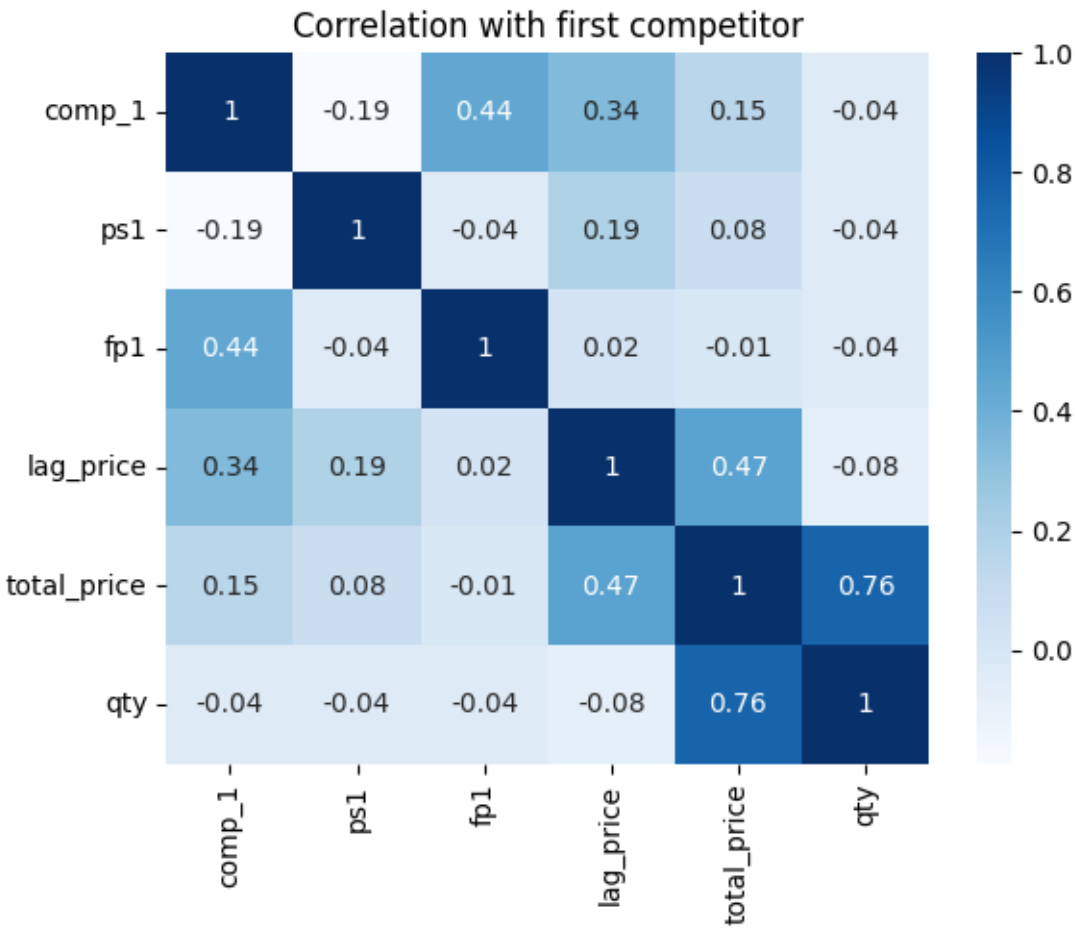
[48]:
```
sns.heatmap(df[set1].corr().round(2),annot=True,cmap='Blues')
plt.title('Heatmap of first set of numerical columns')
plt.show()
```



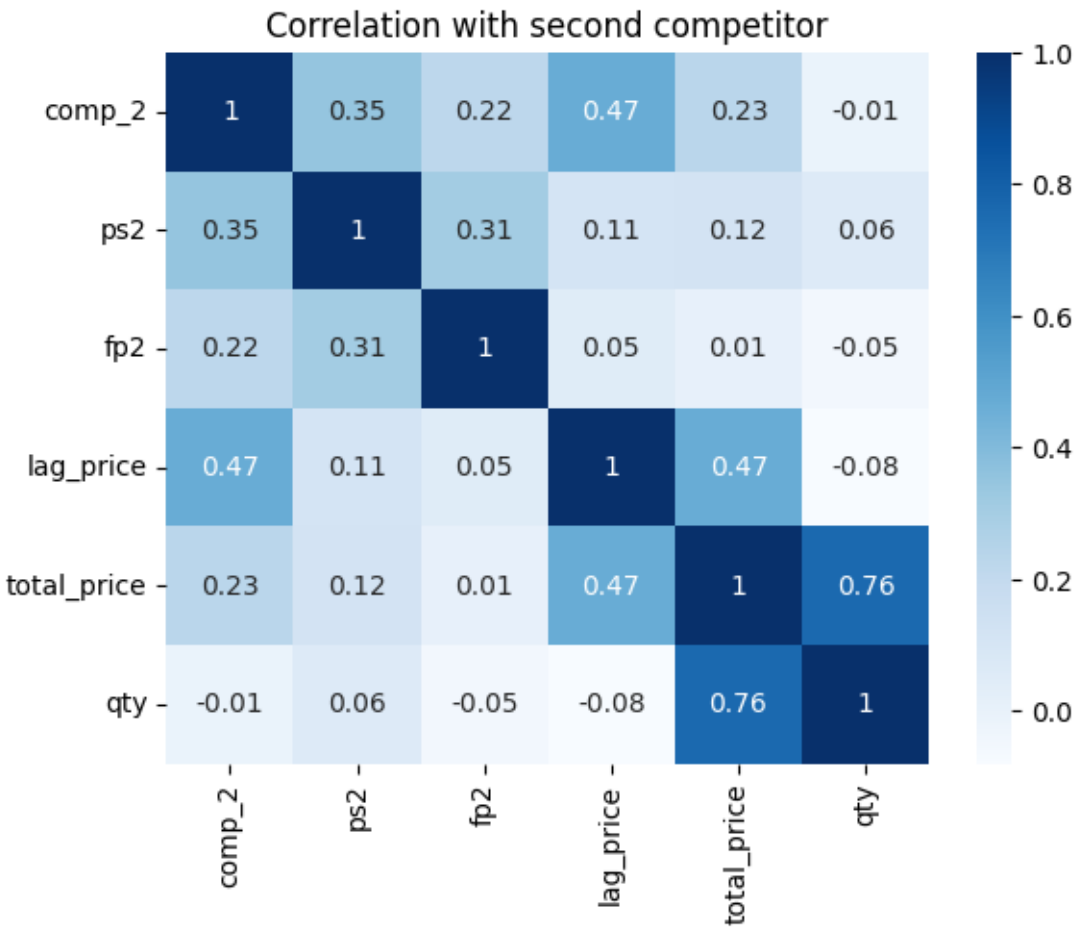
[49]:
```
sns.heatmap(df[set2].corr().round(2),annot=True,cmap='Blues')
plt.title('Heatmap of second set of numerical columns')
plt.show()
```



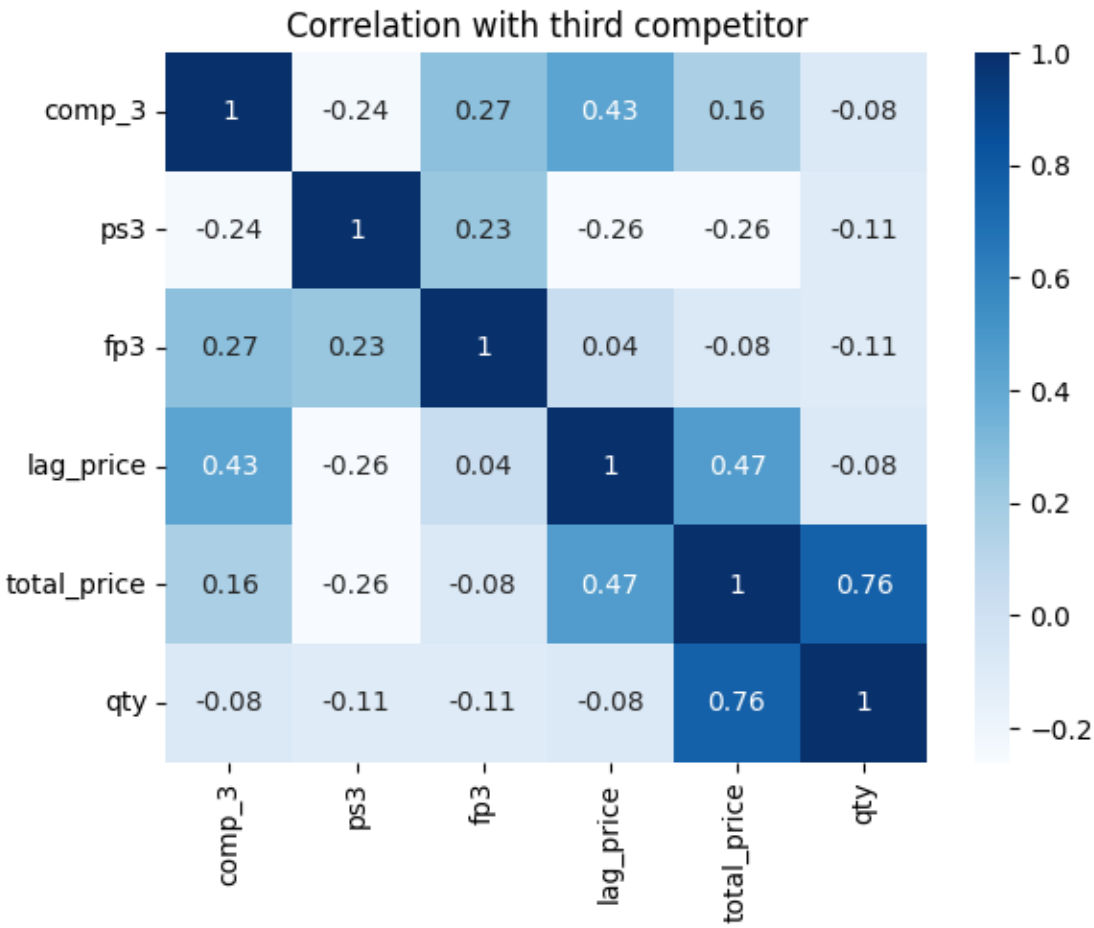
[50]:
```
sns.heatmap(df[set3].corr().round(2),annot=True,cmap='Blues')
plt.title('Correlation with first competitor')
plt.show()
```



[51]:
```
sns.heatmap(df[set4].corr().round(2),annot=True,cmap='Blues')
plt.title('Correlation with second competitor')
plt.show()
```



[52]:
```
sns.heatmap(df[set5].corr().round(2),annot=True,cmap='Blues')
plt.title('Correlation with third competitor')
plt.show()
```



0.4.1 Freight price is positively correlated with weight, total price is positively correlated with quantity, weekday and weekend are negatively correlated. These are quite self explanatory.

[53]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 676 entries, 0 to 675
Data columns (total 30 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   product_id                  676 non-null    object
 1   product_category_name       676 non-null    object
 2   month_year                  676 non-null    object
 3   qty                         676 non-null    float64
 4   total_price                 676 non-null    float64
 5   freight_price               676 non-null    float64
 6   unit_price                  676 non-null    float64
```

```
 7   product_name_lenght         676 non-null    int64
 8   product_description_lenght  676 non-null    int64
 9   product_photos_qty          676 non-null    float64
 10  product_weight_g            676 non-null    float64
 11  product_score               676 non-null    float64
 12  customers                   676 non-null    float64
 13  weekday                     676 non-null    int64
 14  weekend                     676 non-null    int64
 15  holiday                     676 non-null    int64
 16  month                       676 non-null    int64
 17  year                        676 non-null    int64
 18  s                           676 non-null    float64
 19  volume                      676 non-null    int64
 20  comp_1                      676 non-null    float64
 21  ps1                         676 non-null    float64
 22  fp1                         676 non-null    float64
 23  comp_2                      676 non-null    float64
 24  ps2                         676 non-null    float64
 25  fp2                         676 non-null    float64
 26  comp_3                      676 non-null    float64
 27  ps3                         676 non-null    float64
 28  fp3                         676 non-null    float64
 29  lag_price                   676 non-null    float64
dtypes: float64(19), int64(8), object(3)
memory usage: 158.6+ KB
```

0.5 Feature Engineering

[54]:
```
df['Revenue']=df.total_price-df.freight_price

w1=df.comp_1.corr(df.lag_price)
w2=df.comp_2.corr(df.lag_price)
w3=df.comp_3.corr(df.lag_price)
#comp price is weighted mean with correlation of lag_price with respective␣
↪competitive price
df['comp']=(df.comp_1*w1+df.comp_2*w2+df.comp_3*w3)/(w1+w2+w3)
df['comp_ps']=(df.ps1*w1+df.ps2*w2+df.ps3*w3)/(w1+w2+w3)
df['comp_fp']=(df.fp1*w1+df.fp2*w2+df.fp3*w3)/(w1+w2+w3)
df=df.
↪drop(columns=['unit_price','product_name_lenght','product_description_lenght','comp_1','com
```

[55]:
```
df.columns
```

[55]:
```
Index(['product_id', 'product_category_name', 'month_year', 'qty',
       'total_price', 'freight_price', 'product_photos_qty',
       'product_weight_g', 'product_score', 'customers', 'weekday', 'weekend',
       'holiday', 'month', 'year', 's', 'volume', 'lag_price', 'Revenue',
```

```
       'comp', 'comp_ps', 'comp_fp'],
      dtype='object')
```

[56]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 676 entries, 0 to 675
Data columns (total 22 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   product_id             676 non-null    object
 1   product_category_name  676 non-null    object
 2   month_year             676 non-null    object
 3   qty                    676 non-null    float64
 4   total_price            676 non-null    float64
 5   freight_price          676 non-null    float64
 6   product_photos_qty     676 non-null    float64
 7   product_weight_g       676 non-null    float64
 8   product_score          676 non-null    float64
 9   customers              676 non-null    float64
 10  weekday                676 non-null    int64
 11  weekend                676 non-null    int64
 12  holiday                676 non-null    int64
 13  month                  676 non-null    int64
 14  year                   676 non-null    int64
 15  s                      676 non-null    float64
 16  volume                 676 non-null    int64
 17  lag_price              676 non-null    float64
 18  Revenue                676 non-null    float64
 19  comp                   676 non-null    float64
 20  comp_ps                676 non-null    float64
 21  comp_fp                676 non-null    float64
dtypes: float64(13), int64(6), object(3)
memory usage: 116.3+ KB
```

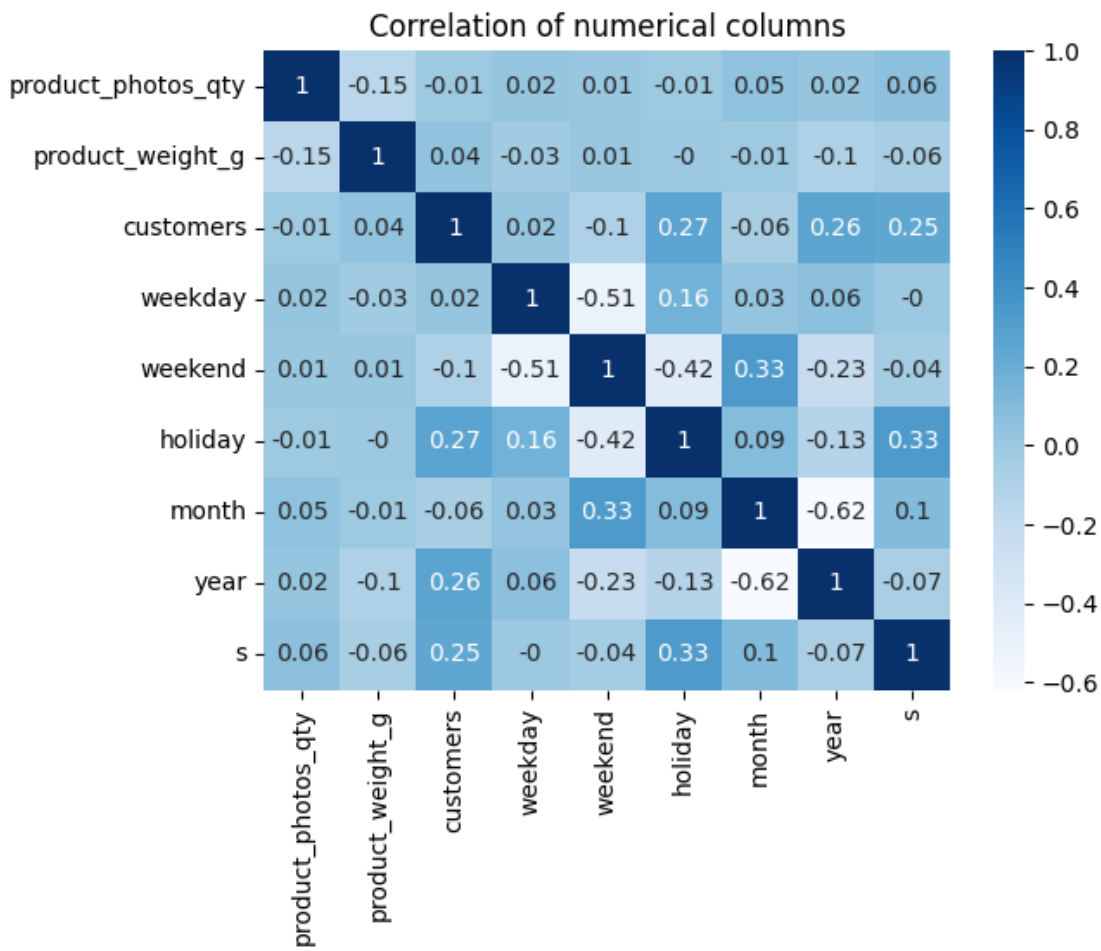
[57]:
```
df.head()
```

[57]:
```
  product_id product_category_name  month_year  qty  total_price  \
0       bed1        bed_bath_table  01-05-2017  1.0        45.95
1       bed1        bed_bath_table  01-06-2017  3.0       137.85
2       bed1        bed_bath_table  01-07-2017  6.0       275.70
3       bed1        bed_bath_table  01-08-2017  4.0       183.80
4       bed1        bed_bath_table  01-09-2017  2.0        91.90

   freight_price  product_photos_qty  product_weight_g  product_score  \
0      15.100000                 2.0             350.0            4.0
1      12.933333                 2.0             350.0            4.0
2      14.840000                 2.0             350.0            4.0
```

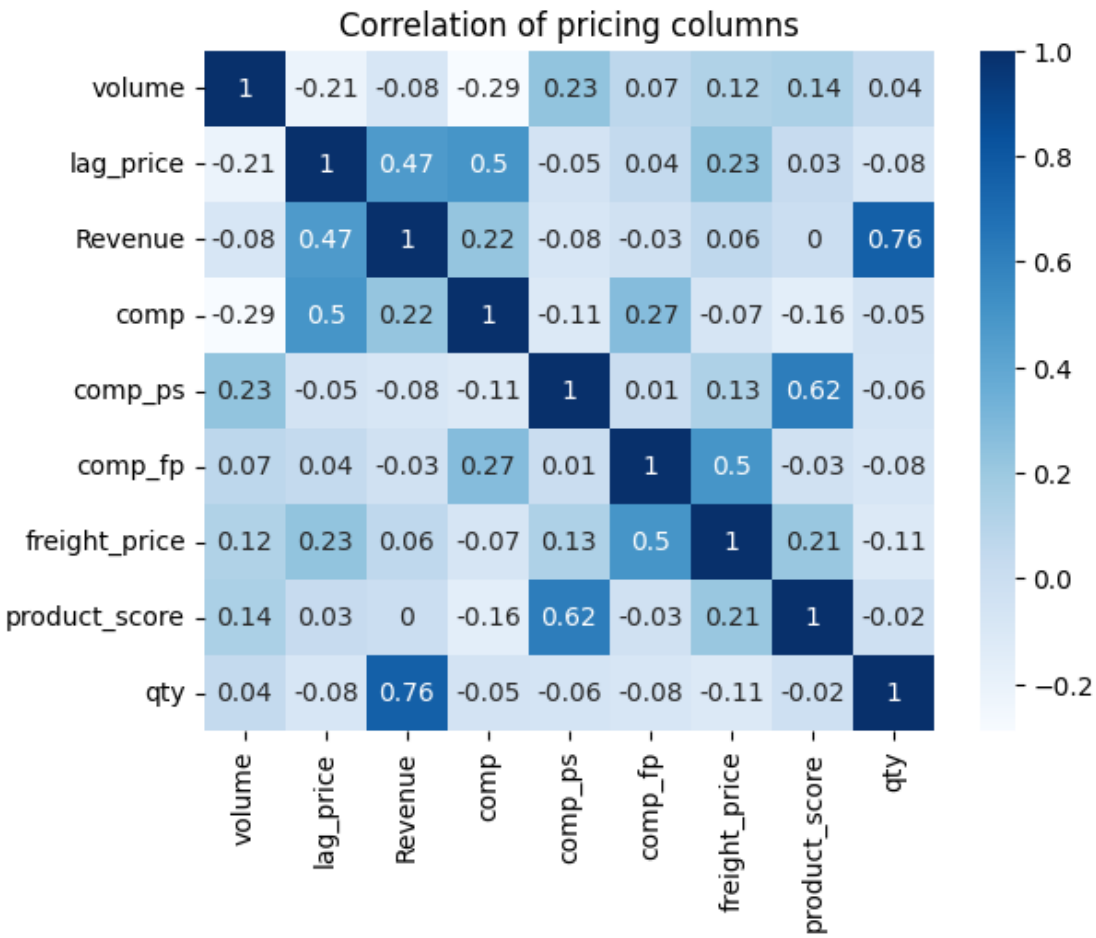
```
3      14.287500                 2.0             350.0            4.0
4      15.100000                 2.0             350.0            4.0

   customers  weekday  weekend  holiday  month  year          s  volume  \
0       57.0       23        8        1      5  2017  10.267394    3800
1       61.0       22        8        1      6  2017   6.503115    3800
2      123.0       21       10        1      7  2017  12.071651    3800
3       90.0       23        8        1      8  2017   9.293873    3800
4       54.0       21        9        1      9  2017   5.555556    3800

   lag_price     Revenue        comp   comp_ps    comp_fp
0      45.90   30.850000  122.099156  4.124377  12.670932
1      45.95  124.916667  119.823163  4.124377  16.617652
2      45.95  260.860000  118.305834  4.124377  17.398863
3      45.95  169.512500  116.223226  4.124377  16.332738
4      45.95   76.800000  102.525124  4.124377  19.604977
```

[58]:
```
set1=['product_photos_qty', 'product_weight_g','customers',
       'weekday', 'weekend', 'holiday', 'month', 'year', 's']

set2=['volume', 'lag_price', 'Revenue', 'comp', 'comp_ps', 'comp_fp',␣
 ↪'freight_price', 'product_score', 'qty']
```

[59]:
```
sns.heatmap(df[set1].corr().round(2),annot=True,cmap='Blues')
plt.title('Correlation of numerical columns')
plt.show()
```



[60]:
```
sns.heatmap(df[set2].corr().round(2),annot=True,cmap='Blues')
plt.title('Correlation of pricing columns')
plt.show()
```



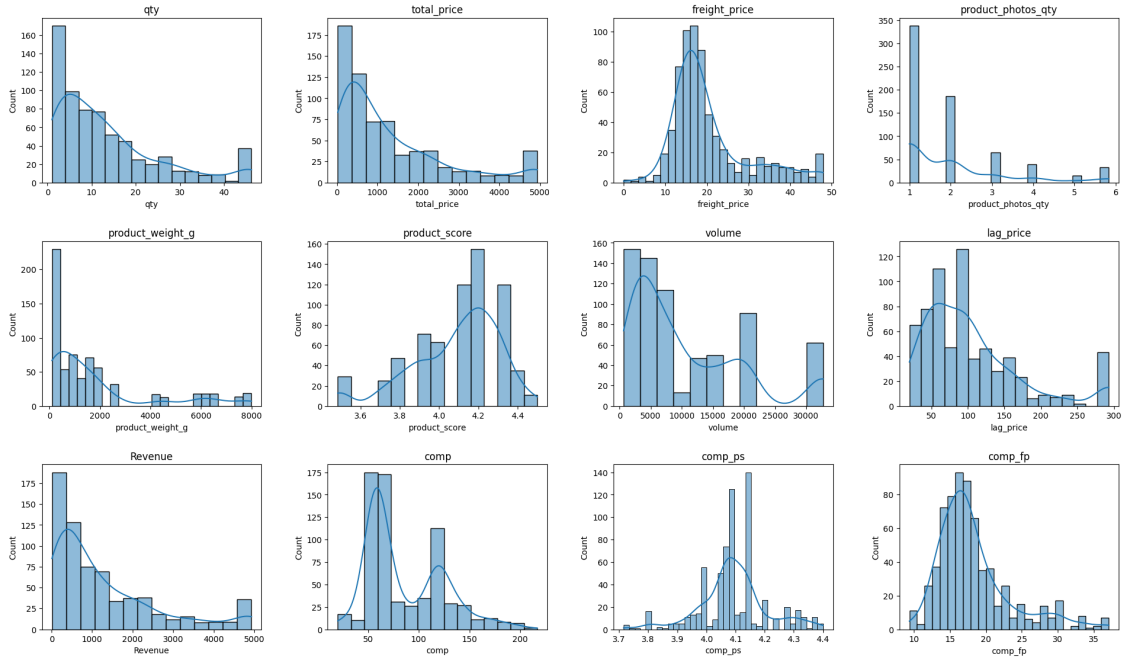
0.5.1 lag_price is mildly positively correlated Revenue, Comp and total_price.

0.5.2 product_score is positively correlated with comp_ps.

0.6 Univariate Analysis

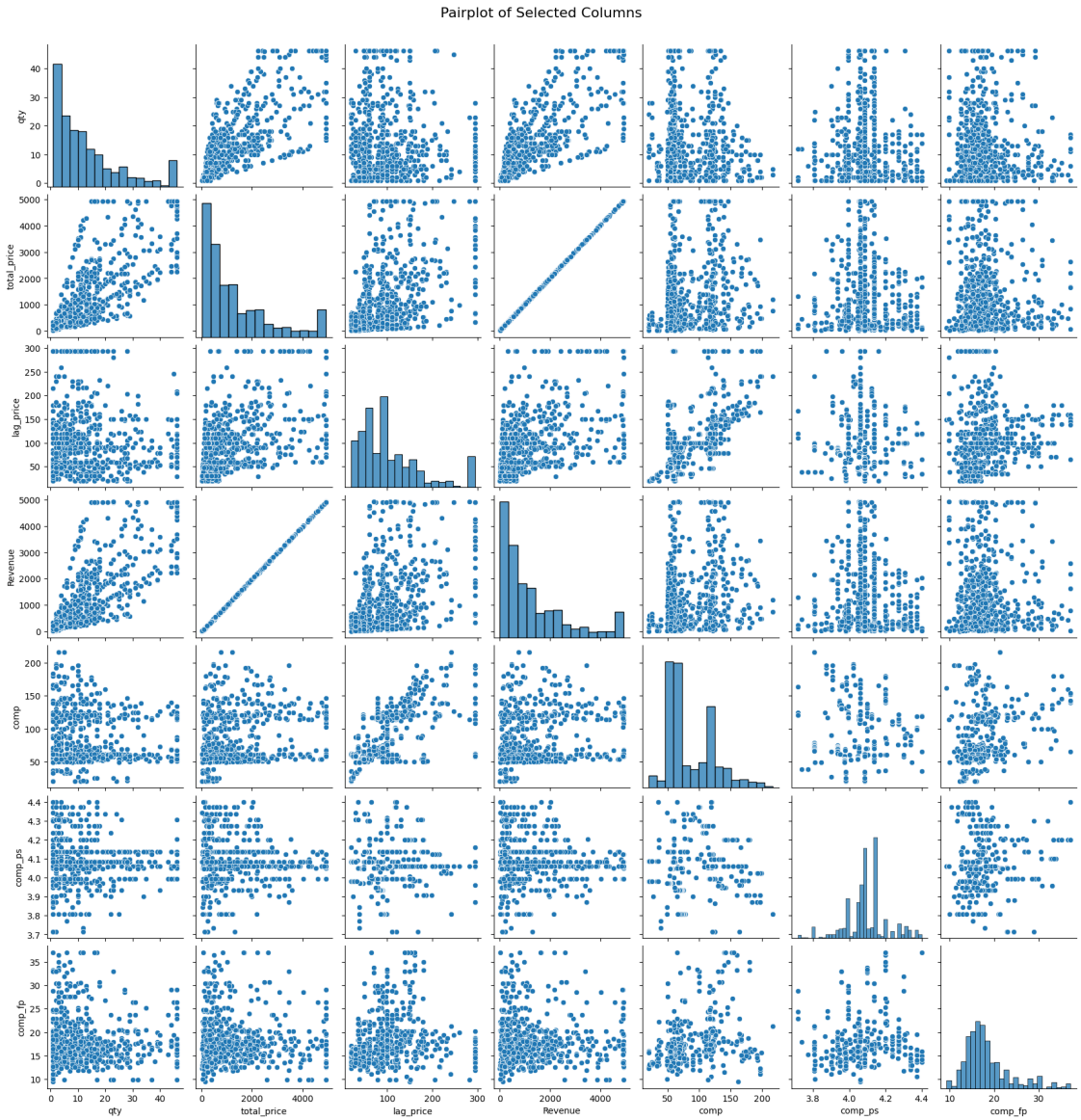
[61]:
```
fig, axes = plt.subplots(3, 4, figsize=(20, 12))
fig.tight_layout(pad=5.0)
axes = axes.flatten()
columns = ['qty', 'total_price', 'freight_price', 'product_photos_qty',
           'product_weight_g', 'product_score', 'volume', 'lag_price',␣
 ↪'Revenue',
           'comp', 'comp_ps', 'comp_fp']
for i, col in enumerate(columns):
    sns.histplot(df[col], kde=True, ax=axes[i])
    axes[i].set_title(col)
for i in range(len(columns), len(axes)):
    axes[i].axis('off')
```

```
plt.show()
```



0.7 Pairplot for important numerical columns

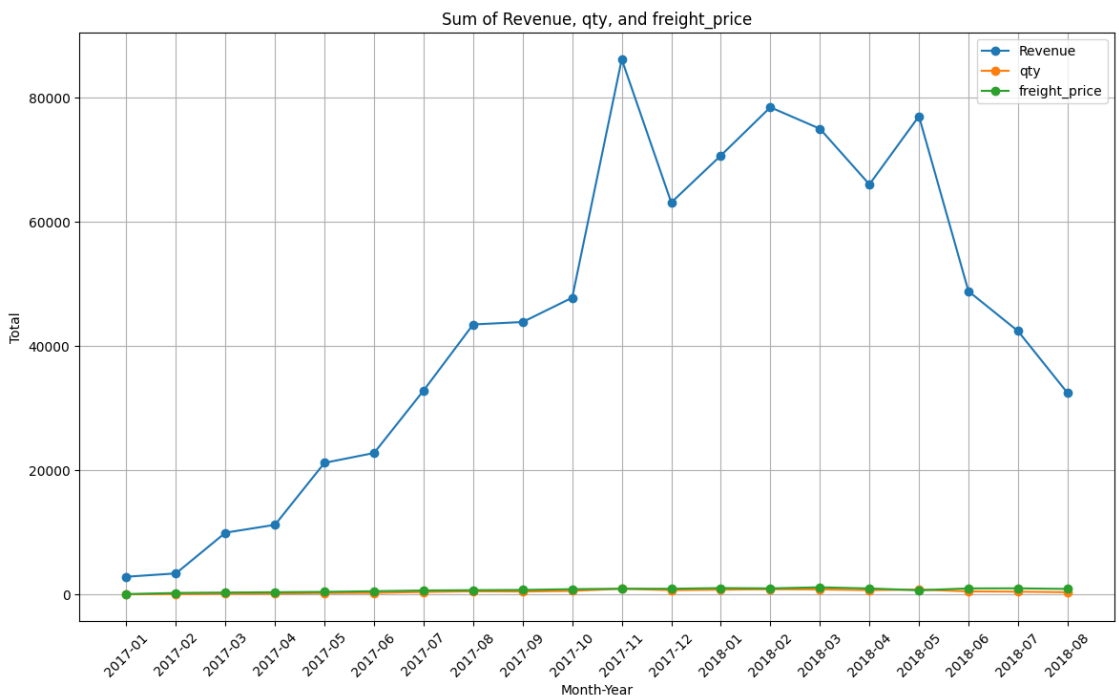
[62]:
```
columns = ['qty', 'total_price', 'lag_price', 'Revenue', 'comp', 'comp_ps',␣
↪'comp_fp']
pairplot = sns.pairplot(df[columns])
plt.suptitle('Pairplot of Selected Columns', y=1.02, fontsize=16)
plt.show()
```



0.8 Temporal Analysis

[63]:
```
monthly_summary = df.groupby(['year', 'month'])[['Revenue', 'qty',␣
↪'freight_price']].sum().reset_index()
monthly_summary['month_year'] = monthly_summary['year'].astype(str) + '-' +␣
↪monthly_summary['month'].astype(str).str.zfill(2)

plt.figure(figsize=(14, 8))
for column in ['Revenue', 'qty', 'freight_price']:
    plt.plot(monthly_summary['month_year'], monthly_summary[column],␣
↪marker='o', label=column)
```

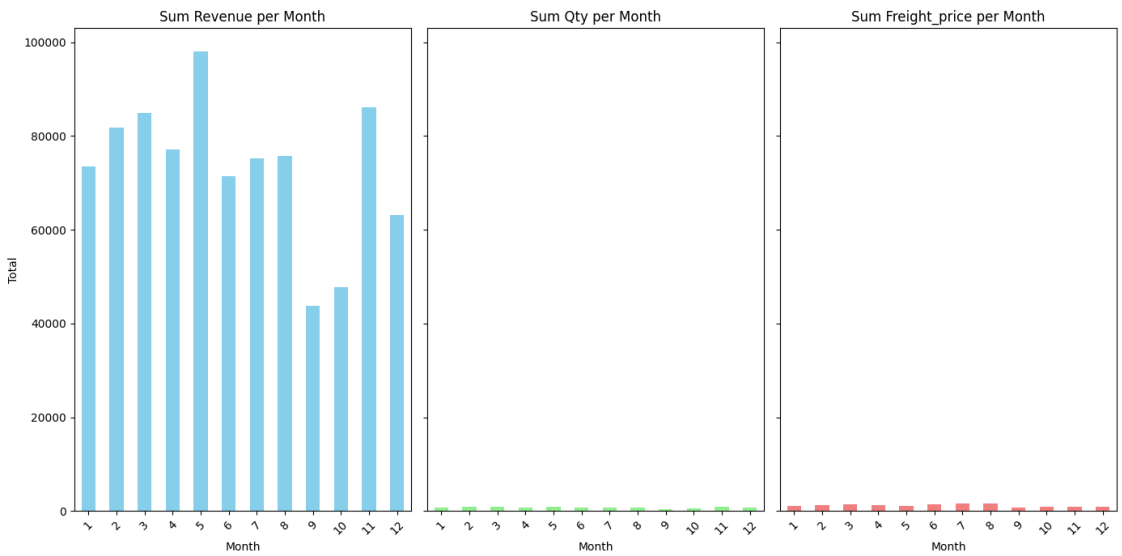
```
plt.title('Sum of Revenue, qty, and freight_price')
plt.xlabel('Month-Year')
plt.ylabel('Total')
plt.xticks(rotation=45)
plt.legend()
plt.grid(visible=True)
plt.show()
```



0.8.1 Revenue peaked in 2017-11, freight_price and qty more or less stays same throughout.

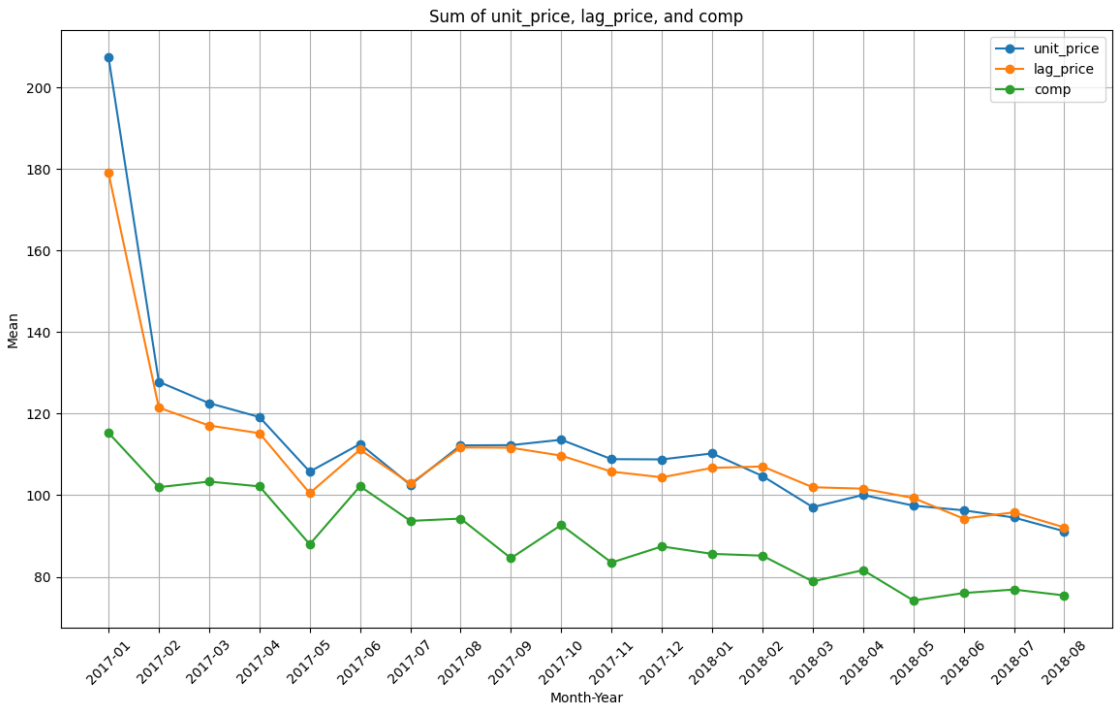
[64]:
```
monthly_means = df.groupby('month')[['Revenue', 'qty', 'freight_price']].sum()

fig, axes = plt.subplots(1, 3, figsize=(14, 7), sharey=True)

colors = ['skyblue', 'lightgreen', 'lightcoral']
columns = ['Revenue', 'qty', 'freight_price']

for i, col in enumerate(columns):
    monthly_means[col].plot(kind='bar', stacked=True, ax=axes[i],␣
 ↪color=colors[i])
    axes[i].set_title(f'Sum {col.capitalize()} per Month')
    axes[i].set_xlabel('Month')
```

```
    axes[i].set_ylabel('Total')
    axes[i].set_xticks(range(0, 12))
    axes[i].set_xticklabels(monthly_means.index, rotation=45)

plt.tight_layout()
plt.show()
```

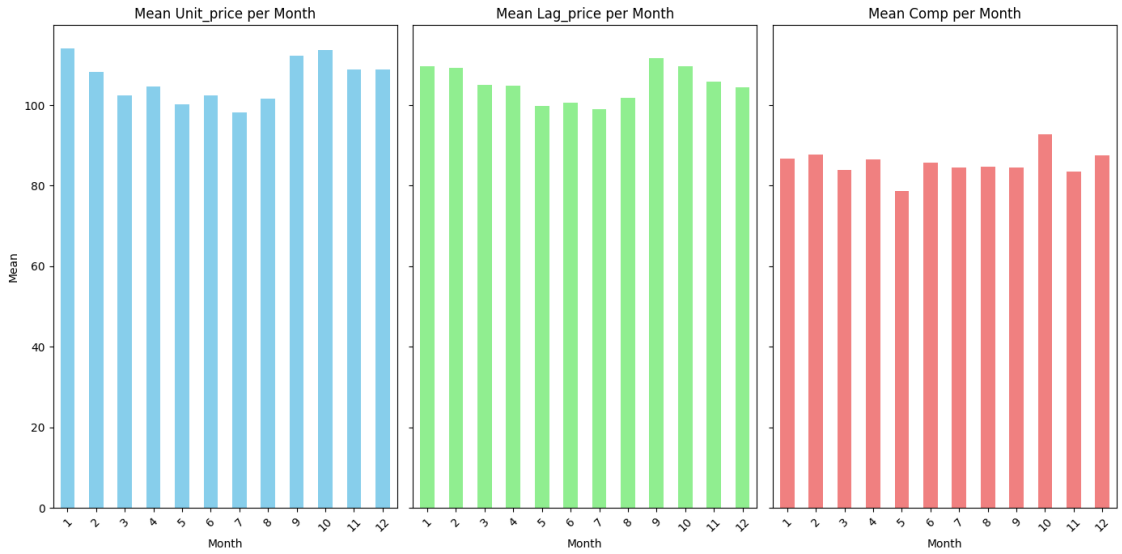


0.8.2 May gives most revenue followed by November then March.

[65]:
```
df_=df.copy()
df_['unit_price']=df_.total_price/df_.qty
monthly_summary = df_.groupby(['year', 'month'])[['unit_price', 'lag_price',␣
 ↪'comp']].mean().reset_index()
monthly_summary['month_year'] = monthly_summary['year'].astype(str) + '-' +␣
 ↪monthly_summary['month'].astype(str).str.zfill(2)

plt.figure(figsize=(14, 8))
for column in ['unit_price', 'lag_price', 'comp']:
    plt.plot(monthly_summary['month_year'], monthly_summary[column],␣
 ↪marker='o', label=column)

plt.title('Sum of unit_price, lag_price, and comp')
plt.xlabel('Month-Year')
plt.ylabel('Mean')
plt.xticks(rotation=45)
plt.legend()
plt.grid(visible=True)
plt.show()
```

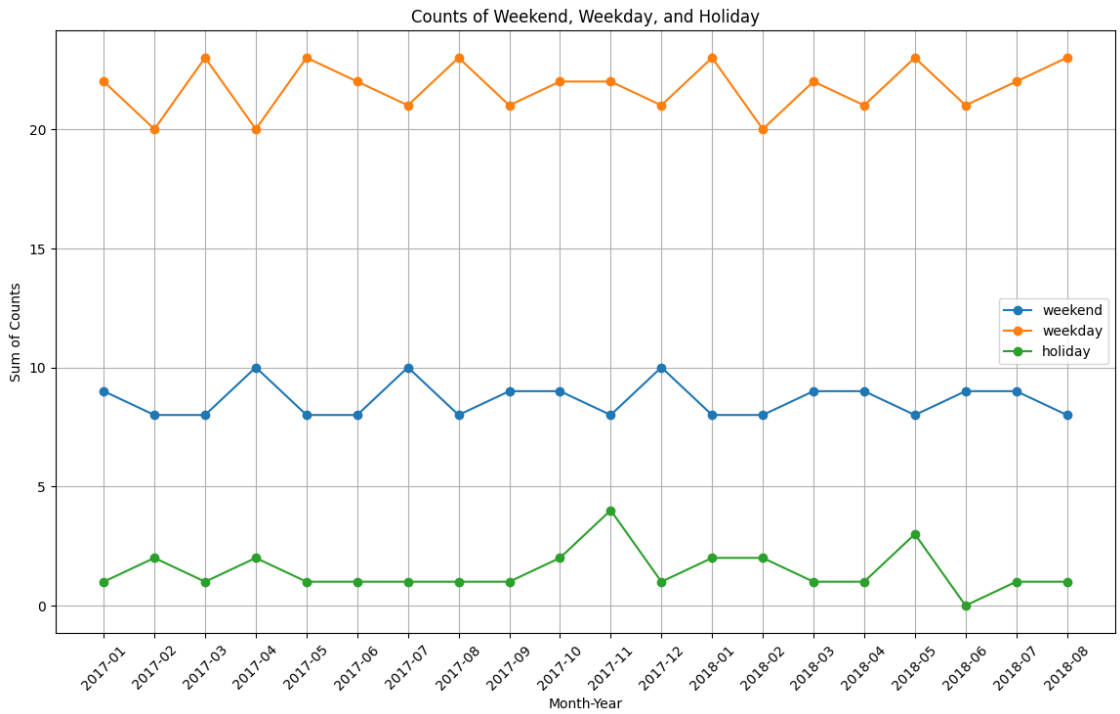


[66]:
```
df_=df.copy()
df_['unit_price']=df_.total_price/df_.qty
monthly_means = df_.groupby('month')[['unit_price', 'lag_price', 'comp']].mean()

fig, axes = plt.subplots(1, 3, figsize=(14, 7), sharey=True)

colors = ['skyblue', 'lightgreen', 'lightcoral']
columns = ['unit_price', 'lag_price', 'comp']

for i, col in enumerate(columns):
    monthly_means[col].plot(kind='bar', stacked=True, ax=axes[i],␣
 ↪color=colors[i])
    axes[i].set_title(f'Mean {col.capitalize()} per Month')
    axes[i].set_xlabel('Month')
    axes[i].set_ylabel('Mean')
    axes[i].set_xticks(range(0, 12))
    axes[i].set_xticklabels(monthly_means.index, rotation=45)

plt.tight_layout()
plt.show()
```

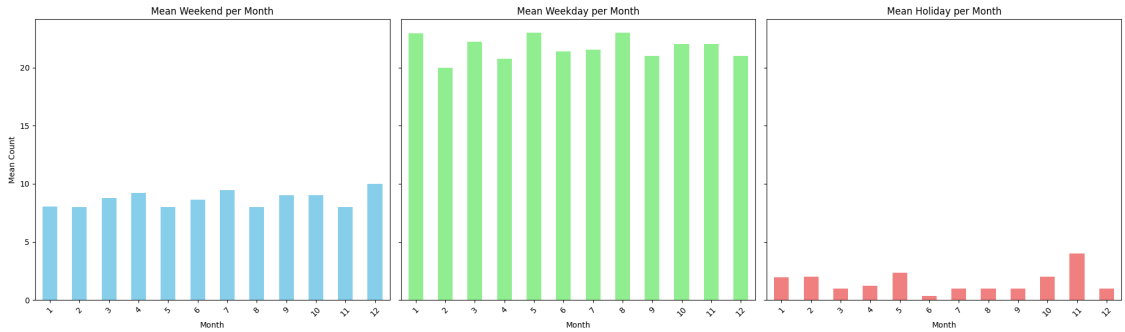


0.8.3 unit_price is always higher than comp price more closer to lag_price maybe more discounts can be given to attract more customers.

[67]:
```
monthly_summary = df.groupby(['year', 'month'])[['weekend', 'weekday',␣
↪'holiday']].mean().reset_index()
monthly_summary['month_year'] = monthly_summary['year'].astype(str) + '-' +␣
↪monthly_summary['month'].astype(str).str.zfill(2)

plt.figure(figsize=(14, 8))
for column in ['weekend', 'weekday', 'holiday']:
    plt.plot(monthly_summary['month_year'], monthly_summary[column],␣
↪marker='o', label=column)

plt.title('Counts of Weekend, Weekday, and Holiday')
plt.xlabel('Month-Year')
plt.ylabel('Sum of Counts')
plt.xticks(rotation=45)
plt.legend()
plt.grid(visible=True)
plt.show()
```



[68]:
```
monthly_means = df.groupby('month')[['weekend', 'weekday', 'holiday']].mean()

fig, axes = plt.subplots(1, 3, figsize=(20, 6), sharey=True)

colors = ['skyblue', 'lightgreen', 'lightcoral']
columns = ['weekend', 'weekday', 'holiday']

for i, col in enumerate(columns):
    monthly_means[col].plot(kind='bar', stacked=True, ax=axes[i],␣
 ↪color=colors[i])
    axes[i].set_title(f'Mean {col.capitalize()} per Month')
    axes[i].set_xlabel('Month')
    axes[i].set_ylabel('Mean Count')
    axes[i].set_xticks(range(0, 12))
    axes[i].set_xticklabels(monthly_means.index, rotation=45)

plt.tight_layout()
plt.show()
```



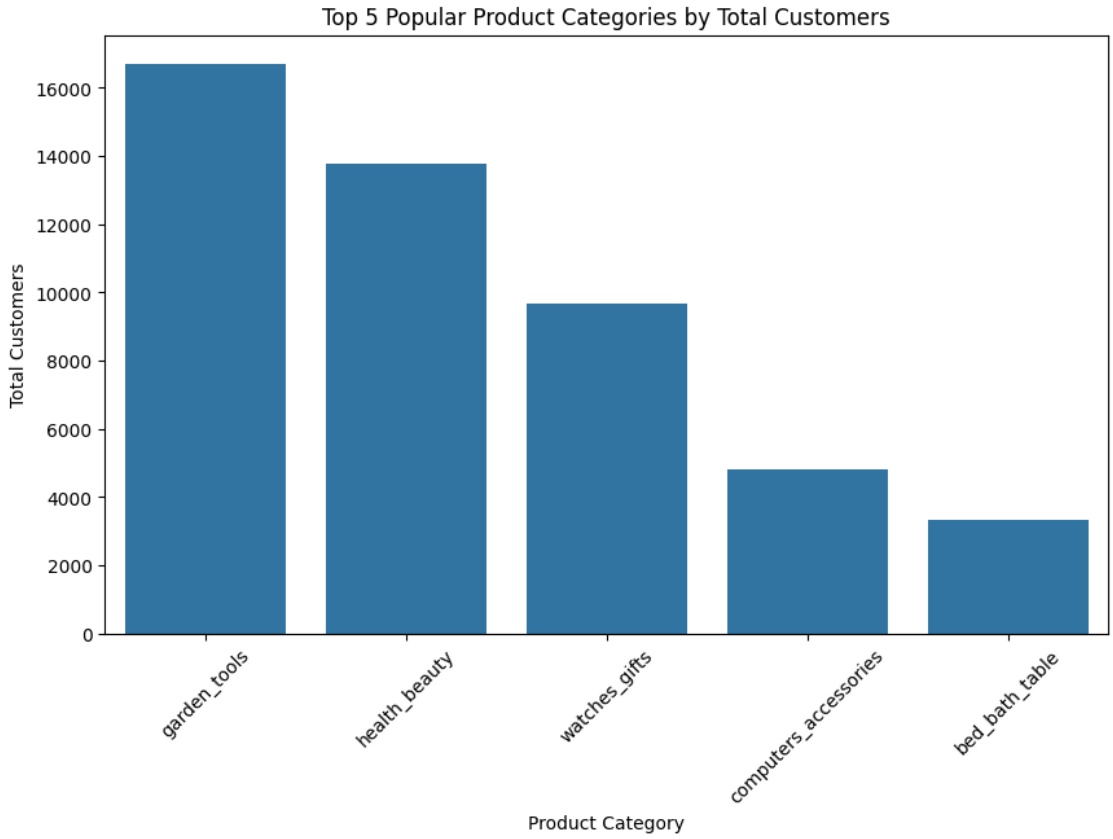
0.8.4 Not much difference between mean `weekday` , `weekend` and `holiday` counts for year-month as well as month levels.

0.9 Popular product categories

[69]:
```
category_summary = df.groupby('product_category_name')['customers'].sum().
↪sort_values(ascending=False)
print(tabulate(category_summary.reset_index(), headers=['Product Category',␣
↪'Total Customers'], tablefmt='grid'))

category_summary=category_summary.head(5)
plt.figure(figsize=(10, 6))
sns.barplot(x=category_summary.index, y=category_summary.values)
plt.title('Top 5 Popular Product Categories by Total Customers')
plt.xlabel('Product Category')
plt.ylabel('Total Customers')
plt.xticks(rotation=45)
plt.show()
```

```
+----+-----------------------+-------------------+
|    | Product Category      |   Total Customers |
+====+=======================+===================+
|  0 | garden_tools          |          16713.8  |
+----+-----------------------+-------------------+
|  1 | health_beauty         |          13765    |
+----+-----------------------+-------------------+
|  2 | watches_gifts         |           9651.62 |
+----+-----------------------+-------------------+
|  3 | computers_accessories |           4807    |
+----+-----------------------+-------------------+
|  4 | bed_bath_table        |           3313    |
+----+-----------------------+-------------------+
|  5 | furniture_decor       |           2495    |
+----+-----------------------+-------------------+
|  6 | cool_stuff            |           2168    |
```

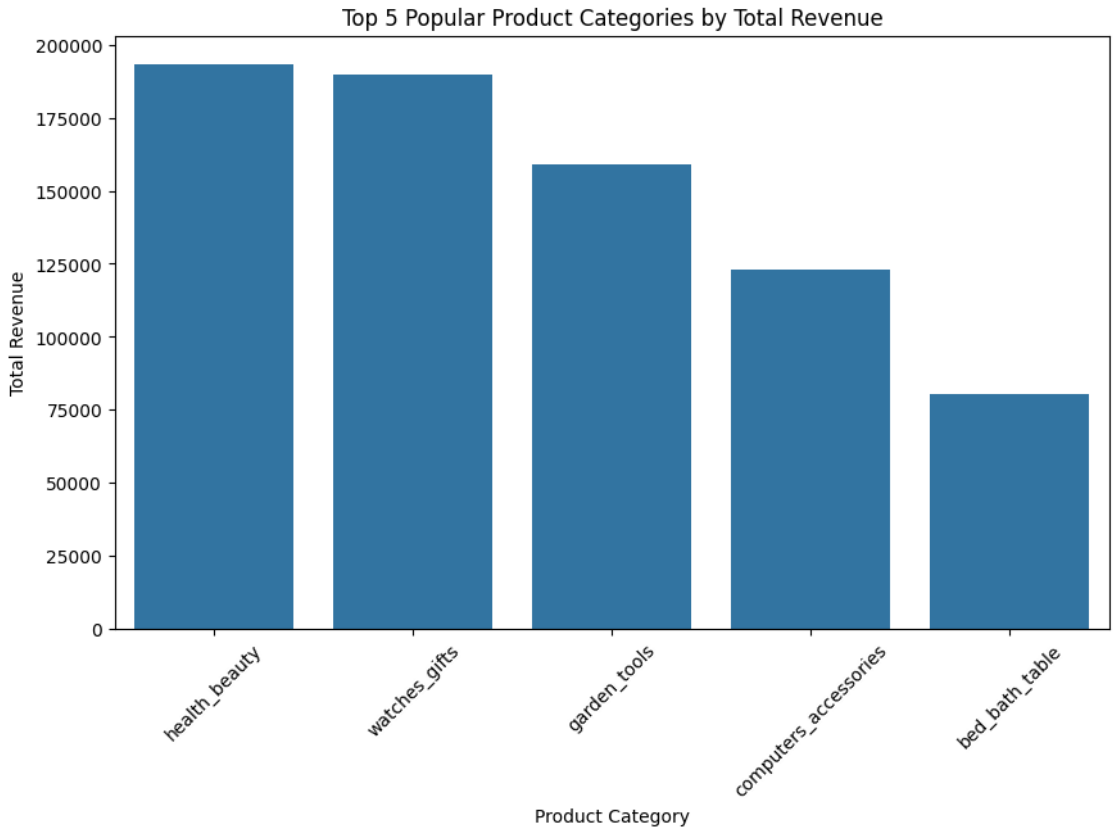
```
+----+-----------------------+-------------------+
|  7 | consoles_games        |            439    |
+----+-----------------------+-------------------+
|  8 | perfumery             |            427    |
+----+-----------------------+-------------------+
```



[70]:
```
category_summary = df.groupby('product_category_name')['Revenue'].sum().
 ↪sort_values(ascending=False)
print(tabulate(category_summary.reset_index(), headers=['Product Category',␣
 ↪'Total Revenue'], tablefmt='grid'))

category_summary=category_summary.head(5)
plt.figure(figsize=(10, 6))
sns.barplot(x=category_summary.index, y=category_summary.values)
plt.title('Top 5 Popular Product Categories by Total Revenue')
plt.xlabel('Product Category')
plt.ylabel('Total Revenue')
plt.xticks(rotation=45)
plt.show()
```

```
+----+-----------------------+-----------------+
```

	Product Category	Total Revenue
0	health_beauty	193556
1	watches_gifts	189858
2	garden_tools	159146
3	computers_accessories	123182
4	bed_bath_table	80450.3
5	cool_stuff	56884
6	furniture_decor	50131.8
7	perfumery	19939.8
8	consoles_games	5474.89

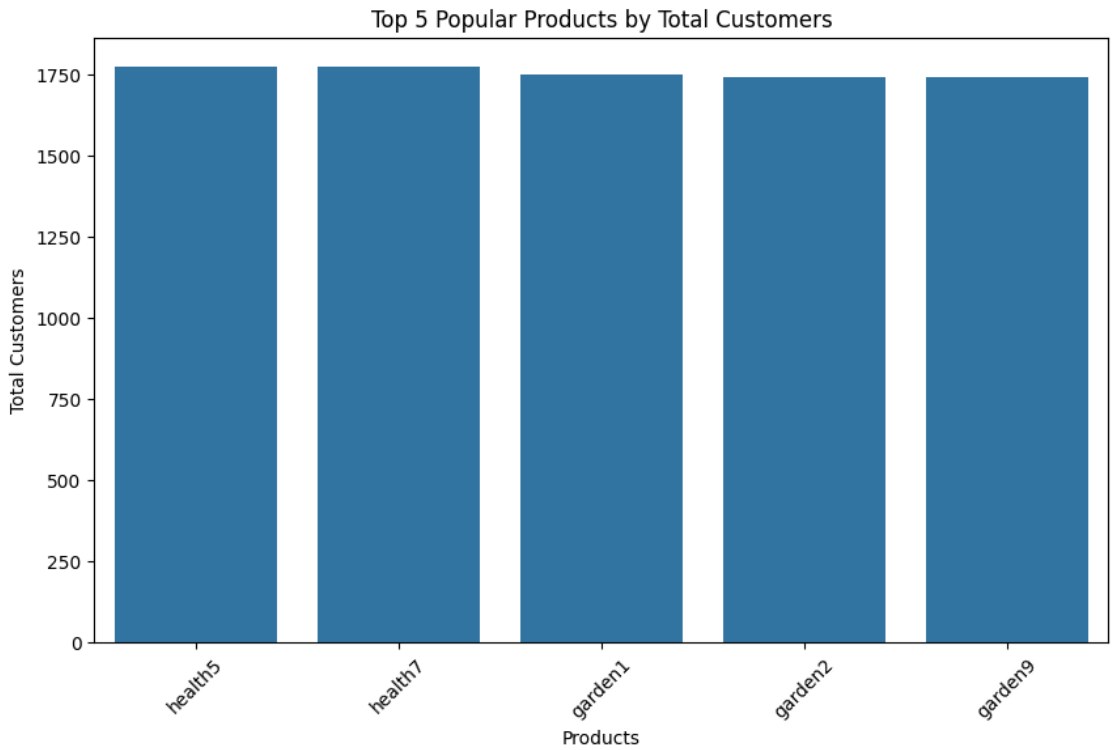


health_beauty, watches_gifts and garden_tools are the categories that are very important.

0.10 Popular product_id s

[71]:
```
product_summary = df.groupby('product_id')['customers'].sum().
 ↪sort_values(ascending=False).head()
print(tabulate(product_summary.reset_index(), headers=['Product Category',␣
 ↪'Total Customers'], tablefmt='grid'))

plt.figure(figsize=(10, 6))
sns.barplot(x=product_summary.index, y=product_summary.values)
plt.title('Top 5 Popular Products by Total Customers')
plt.xlabel('Products')
plt.ylabel('Total Customers')
plt.xticks(rotation=45)
plt.show()
```

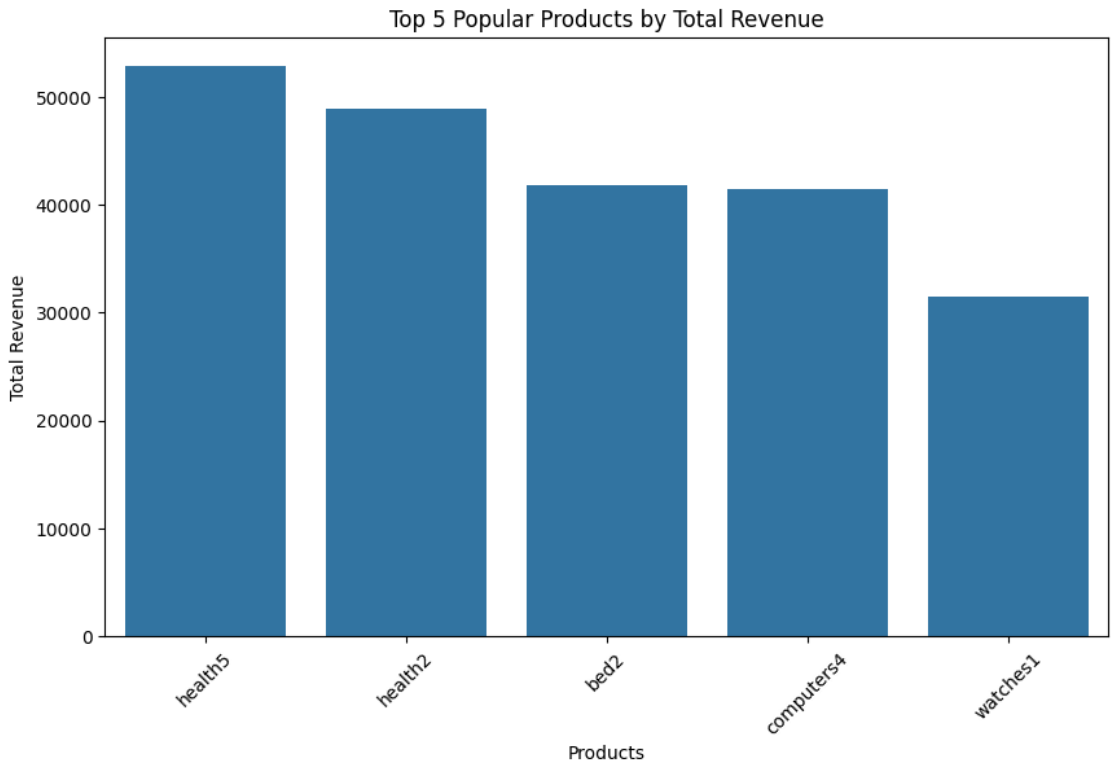
```
+----+--------------------+--------------------+
|    | Product Category   |   Total Customers |
+====+====================+====================+
|  0 | health5            |           1775    |
+----+--------------------+--------------------+
|  1 | health7            |           1775    |
+----+--------------------+--------------------+
|  2 | garden1            |           1748.58 |
+----+--------------------+--------------------+
|  3 | garden2            |           1741.58 |
+----+--------------------+--------------------+
|  4 | garden9            |           1741.58 |
+----+--------------------+--------------------+
```



[72]:
```
product_summary = df.groupby('product_id')['Revenue'].sum().
 ↪sort_values(ascending=False).head(5)
print(tabulate(product_summary.reset_index(), headers=['Product Category',␣
 ↪'Total Revenue'], tablefmt='grid'))

plt.figure(figsize=(10, 6))
sns.barplot(x=product_summary.index, y=product_summary.values)
plt.title('Top 5 Popular Products by Total Revenue')
plt.xlabel('Products')
plt.ylabel('Total Revenue')
plt.xticks(rotation=45)
plt.show()
```

```
+----+--------------------+-----------------+
|    | Product Category   |   Total Revenue |
+====+====================+=================+
|  0 | health5            |         52851   |
+----+--------------------+-----------------+
|  1 | health2            |         48894.2 |
+----+--------------------+-----------------+
|  2 | bed2               |         41866.6 |
+----+--------------------+-----------------+
|  3 | computers4         |         41485.9 |
```

```
+----+--------------------+-----------------+
|  4 | watches1           |         31473.4 |
+----+--------------------+-----------------+
```



0.10.1 Popular products like `health5`, `health2` , `bed2` and `health7` should be focused on to increase revenue.

1 Time Series predicting `Revenue` per `product_id` using prophet library

[73]:
```
df.month_year=pd.to_datetime(df.month_year)
def forecast_Revenue(df, product_id, periods):
  df_=df[df.product_id==product_id]
  df_ = df_[['month_year', 'Revenue']].rename(columns={'month_year':'ds',␣
 ↪'Revenue':'y'})
  m = Prophet()
  m.add_seasonality(name='monthly', period=30.5, fourier_order=5)
  m.fit(df_)
  future = m.make_future_dataframe(periods=periods)
  forecast = m.predict(future)
  return m, forecast
```

[74]:
```
##Prophet not working in local but working in colab

#model,forecast = forecast_Revenue(df,'bed1',365)
#plot_plotly(model, forecast)
```

```
INFO:prophet:Disabling yearly seasonality. Run prophet with
yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with
daily_seasonality=True to override this.
INFO:prophet:n_changepoints greater than number of observations. Using 11.
DEBUG:cmdstanpy:input tempfile: /tmp/tmp825mfvzq/_elq6i_t.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmp825mfvzq/_h9nph3s.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.10/dist-
packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=85205', 'data',
'file=/tmp/tmp825mfvzq/_elq6i_t.json', 'init=/tmp/tmp825mfvzq/_h9nph3s.json',
'output',
'file=/tmp/tmp825mfvzq/prophet_model0lh5e2ha/prophet_model-20241118165602.csv',
'method=optimize', 'algorithm=newton', 'iter=10000']
16:56:02 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
16:56:03 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
```

1.1 Similarly we can get prediction of `Revenue` or other required columns for all `product_ids` and plan accordingly.

1.2 Using Regression method to predict the `total_price` and `qty' andunit_price‘`

[75]:
```
df=df.drop(columns=['month_year'])
```

[76]:
```
df.columns
```

[76]:
```
Index(['product_id', 'product_category_name', 'qty', 'total_price',
       'freight_price', 'product_photos_qty', 'product_weight_g',
       'product_score', 'customers', 'weekday', 'weekend', 'holiday', 'month',
       'year', 's', 'volume', 'lag_price', 'Revenue', 'comp', 'comp_ps',
       'comp_fp'],
      dtype='object')
```

[77]:
```
le = LabelEncoder()
df.product_id=le.fit_transform(df.product_id)
df.product_category_name=le.fit_transform(df.product_category_name)
```

[78]:
```
all=df.copy()
df.head()
```

[78]:

```
   product_id  product_category_name  qty  total_price  freight_price  \
0           0                      0  1.0        45.95      15.100000
1           0                      0  3.0       137.85      12.933333
2           0                      0  6.0       275.70      14.840000
3           0                      0  4.0       183.80      14.287500
4           0                      0  2.0        91.90      15.100000

   product_photos_qty  product_weight_g  product_score  customers  weekday  \
0                 2.0             350.0            4.0       57.0       23
1                 2.0             350.0            4.0       61.0       22
2                 2.0             350.0            4.0      123.0       21
3                 2.0             350.0            4.0       90.0       23
4                 2.0             350.0            4.0       54.0       21

   weekend  holiday  month  year          s  volume  lag_price     Revenue  \
0        8        1      5  2017  10.267394    3800      45.90   30.850000
1        8        1      6  2017   6.503115    3800      45.95  124.916667
2       10        1      7  2017  12.071651    3800      45.95  260.860000
3        8        1      8  2017   9.293873    3800      45.95  169.512500
4        9        1      9  2017   5.555556    3800      45.95   76.800000

         comp   comp_ps    comp_fp
0  122.099156  4.124377  12.670932
1  119.823163  4.124377  16.617652
2  118.305834  4.124377  17.398863
3  116.223226  4.124377  16.332738
4  102.525124  4.124377  19.604977
```

[79]:

```
X,y=all.drop(columns=['qty','total_price']),all['qty']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=95, stratify=all.product_id)
param_distributions = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [3, 5, 10, 20, 50, None],
    'learning_rate': np.arange(.1, 1, .1),
    'gamma': np.arange(0, 1, .2),
    'subsample': np.arange(.5, 1, .1),
    'colsample_bytree': np.arange(.5, 1, .1),
    'reg_alpha': np.arange(0, 1, .2),
    'reg_lambda': np.arange(0, 1, .2)
}
xgb_model=XGBRegressor(random_state=95)
rs=RandomizedSearchCV(estimator=xgb_model,
                      param_distributions=param_distributions,
                      cv=4, n_iter=100, random_state=95,
                      scoring='f1_macro')
```

```
rs.fit(X_train,y_train)
recognizer=rs.best_estimator_
print(rs.best_params_)
print(f'R^2 score for quantity is {recognizer.score(X_test,y_test)}')
print(f'Rmse for quantity is {np.sqrt(mse(recognizer.predict(X_test),y_test))}')
```

```
{'subsample': 0.8999999999999999, 'reg_lambda': 0.8, 'reg_alpha': 0.8,
'n_estimators': 200, 'max_depth': 3, 'learning_rate': 0.9, 'gamma': 0.4,
'colsample_bytree': 0.8999999999999999}
R^2 score for quantity is 0.8885279187878082
Rmse for quantity is 3.854573933199853
```

[80]:
```
X,y=all.drop(columns=['total_price','qty']),all['total_price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=95, stratify=all.product_id)
param_distributions = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [3, 5, 10, 20, 50, None],
    'learning_rate': np.arange(.1, 1, .1),
    'gamma': np.arange(0, 1, .2),
    'subsample': np.arange(.5, 1, .1),
    'colsample_bytree': np.arange(.5, 1, .1),
    'reg_alpha': np.arange(0, 1, .2),
    'reg_lambda': np.arange(0, 1, .2)
}
xgb_model=XGBRegressor(random_state=95)
rs=RandomizedSearchCV(estimator=xgb_model,
                      param_distributions=param_distributions,
                      cv=4, n_iter=100, random_state=95,
                      scoring='f1_macro')


rs.fit(X_train,y_train)
recognizer=rs.best_estimator_
print(rs.best_params_)
print(f'R^2 score for total_price is {recognizer.score(X_test,y_test)}')
print(f'Rmse for total_price is {np.sqrt(mse(recognizer.
 ↪predict(X_test),y_test))}')
```

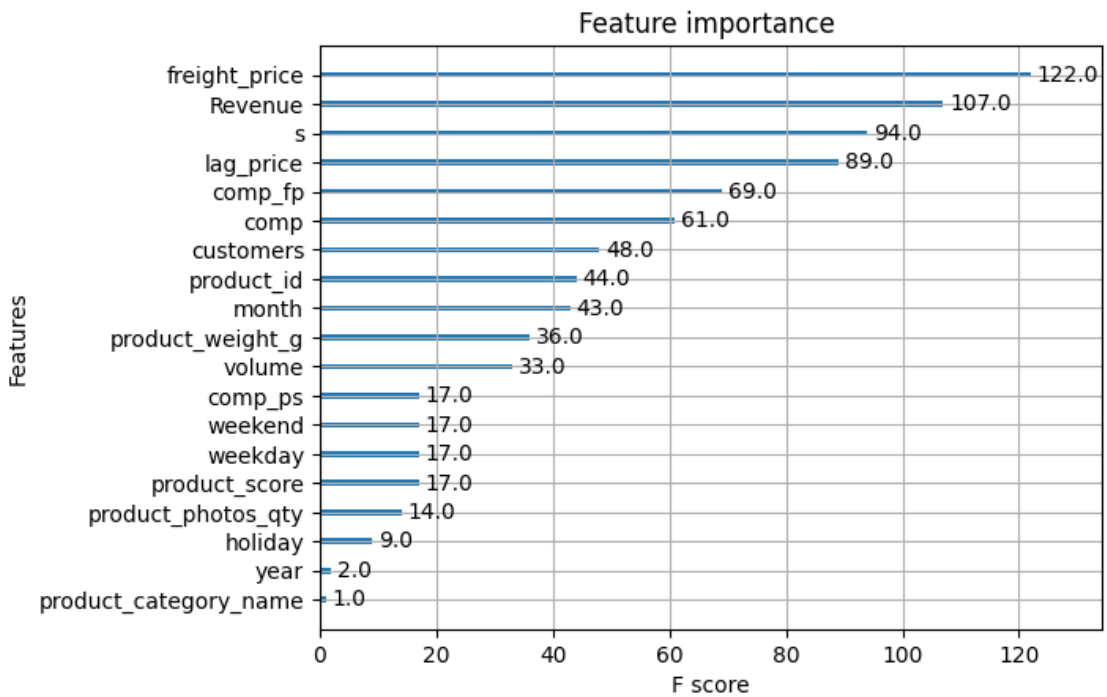
```
{'subsample': 0.8999999999999999, 'reg_lambda': 0.8, 'reg_alpha': 0.8,
'n_estimators': 200, 'max_depth': 3, 'learning_rate': 0.9, 'gamma': 0.4,
'colsample_bytree': 0.8999999999999999}
R^2 score for total_price is 0.9979302303112106
Rmse for total_price is 63.97622652853775
```

[81]:
```
X,y=all.drop(columns=['total_price','qty']),all['total_price']/all['qty']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=95, stratify=all.product_id)
param_distributions = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [3, 5, 10, 20, 50, None],
    'learning_rate': np.arange(.1, 1, .1),
    'gamma': np.arange(0, 1, .2),
    'subsample': np.arange(.5, 1, .1),
    'colsample_bytree': np.arange(.5, 1, .1),
    'reg_alpha': np.arange(0, 1, .2),
    'reg_lambda': np.arange(0, 1, .2)
}
xgb_model=XGBRegressor(random_state=95)
rs=RandomizedSearchCV(estimator=xgb_model,
                      param_distributions=param_distributions,
                      cv=4, n_iter=100, random_state=95,
                      scoring='f1_macro')


rs.fit(X_train,y_train)
recognizer=rs.best_estimator_
print(rs.best_params_)
print(f'R^2 score for unit_price is {recognizer.score(X_test,y_test)}')
print(f'Rmse for unit_price is {np.sqrt(mse(recognizer.
 ↪predict(X_test),y_test))}')
```

```
{'subsample': 0.8999999999999999, 'reg_lambda': 0.8, 'reg_alpha': 0.8,
'n_estimators': 200, 'max_depth': 3, 'learning_rate': 0.9, 'gamma': 0.4,
'colsample_bytree': 0.8999999999999999}
R^2 score for unit_price is 0.8433618568318304
Rmse for unit_price is 28.00764712996934
```

[82]:
```
plot_importance(recognizer)
plt.title('Feature importance ')
plt.show()
```



1.2.1 Use the models to predict prices and quantities based on the different parameters, the R^2 score is good so we can trust the models, also we can compare with some other baseline.

2 Insights and Recommendations

- **Correlations**:
  - Freight price is positively correlated with weight.
  - Total price shows a positive correlation with quantity.
  - Weekday and weekend are negatively correlated.
  - `lag_price` shows mild positive correlation with revenue, competition (`Comp`), and `total_price`.
  - `product_score` is positively correlated with `comp_ps`.
- **Trends and Patterns**:
  - Revenue peaked in **November 2017**, while `freight_price` and `qty` remained stable over time.
  - `unit_price` is consistently higher than competitor prices and is closer to `lag_price`, suggesting room for more competitive pricing (e.g., offering discounts).
- **Comparative Analysis**:
  - No significant differences in mean counts between weekdays, weekends, and holidays at the `year-month` or `month` levels.
- **Important Categories**:
  - **health_beauty**, **watches_gifts**, and **garden_tools** are key product categories for revenue focus.

- High-revenue products like **health5**, **health2**, **bed2**, and **health7** should be prioritized to drive further sales.

2.0.1 Model Performance and Recommendations

- **Regression Models** were used to predict `total_price`, `qty`, and `unit_price` with promising results:
  - **Quantity**: $R^2 = 0.89$, RMSE = 3.85
  - **Total Price**: $R^2 = 0.998$, RMSE = 63.98
  - **Unit Price**: $R^2 = 0.84$, RMSE = 28.01

2.0.2 Recommendations:

1. **Implement targeted discount strategies** for popular products to attract more customers and increase revenue.
2. **Focus marketing efforts** on high-revenue categories and products (e.g., **health_beauty** and products like **health5**).
3. **Monitor competitive pricing** and adjust `unit_price` to stay competitive while maintaining profitability.
4. **Leverage the predictive models** to forecast `total_price`, `qty`, and `unit_price` for various scenarios to plan pricing and inventory effectively.
5. Continue evaluating and refining models against baselines to ensure sustained accuracy and performance.